

IoT Lab Assignment 6

RSA Protected Socket Programming

Part 2 – Socket Programming

Used multiple terminal windows.

```
(.venv) PS C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.1 Socket\Problem> python Server.py -p 1234
Server is listening to port 1234
=====
waiting for connection
Got connection from ('127.0.0.1', 54720)
client: Hello Server
client e: 65537
client N: 4611299948341116473
=====
waiting for connection
␣
(.venv) PS C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.1 Socket\Problem> python .\Client.py -p 1234
Trying to connect to 127.0.0.1:1234
Server: Hello, this is the server
Server: 2438923152802212086
```

Part 3 – RSA

Example 1	p	1637842597
	q	2657124709
	N	4351952033941429273
	e	65537
	d	327573422070677585
	m	2146210639644006206
	c	768594581363479181
Example 2	p	1900493143
	q	2394474161
	N	4550681724071178023
	e	65537
	d	4530892222390974593
	m	914533224344639919
	c	3295824207614022858
Example 3	p	1907263091
	q	2387704201
	N	4553980094792945291

	e	65537
	d	1438314751868679473
	m	3135653864232958471
	c	3360043531845750575

```

Example 1
expected N = 4351952033941429273
computed N = 4351952033941429273
expected d = 327573422070677585
computed d = 327573422070677585
expected c = 768594581363479181
computed c = 768594581363479181
expected m = 2146210639644006206
decrypted m = 2146210639644006206
-----
Example 2
expected N = 4550681724071178023
computed N = 4550681724071178023
expected d = 4530892222390974593
computed d = 4530892222390974593
expected c = 3295824207614022858
computed c = 3295824207614022858
expected m = 914533224344639919
decrypted m = 914533224344639919
-----
Example 3
expected N = 4553980094792945291
computed N = 4553980094792945291
expected d = 1438314751868679473
computed d = 1438314751868679473
expected c = 3360043531845750575
computed c = 3360043531845750575
expected m = 3135653864232958471
decrypted m = 3135653864232958471
-----

```

Exercise 1 Undergrad and Graduate Students	p	1998847601
	q	2296119677
	N	4589593307980344877
	e	65537
	d	2006025606665969473
	m	10001
	c	1258515386968475538
Exercise 2 Undergrad and Graduate Students	N	4400141856896539081
	d	2764011776369096961
	m	123456
	c	2503673653585535070
Exercise 3 Graduate Students Only	p	1940829017
	q	2354138273
	N	4568979870268667641
	e	139241
	d	610100779569588057

```

PS C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.2 RSA\Problem> cd "C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.2 RSA\Problem"; @'
>> import Number_Package as npkg
>>
>> # Exercise 1
>> p1 = 1998847601 ...
>> m1 = 10001
>> N1 = p1 * q1
>> phi1 = (p1 - 1) * (q1 - 1)
>> d1 = npkg.mult_inv_mod_N(e1, phi1)
>> c1 = npkg.exp_mod(m1, e1, N1)
>>
>> print('Exercise 1')
>> print('N =', N1)
>> print('d =', d1)
>> print('c =', c1)
>> print('-' * 50)
>>
>> # Exercise 2
>> N2 = 4400141856896539081
>> d2 = 2764011776369096961
>> c2 = 2503673653585535070
>> m2 = npkg.exp_mod(c2, d2, N2)
>>
>> print('Exercise 2')
>> print('m =', m2)
>> print('-' * 50)
>>
>> # Exercise 3
>> p3 = 1940829017
>> N3 = 4568979870268667641
>> d3 = 610100779569588057
>> q3 = N3 // p3
>> phi3 = (p3 - 1) * (q3 - 1)
>> e3 = npkg.mult_inv_mod_N(d3, phi3)
>>
>> print('Exercise 3')
>> print('q =', q3)
>> print('e =', e3)
>> print('-' * 50)
>> '@ | c:/Users/andre/GitHub/IoT/.venv/Scripts/python.exe -

```

```

Exercise 1
N = 4589593307980344877
d = 200602560665969473
c = 1258515386968475538
-----

Exercise 2
m = 123456
-----

Exercise 3
q = 2354138273
e = 139241
-----

PS C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.2 RSA\Problem>

```

Part 4 – RSA Protected Socket

Is it advisable to use RSA for transmitting large amounts of data (for example, encrypting a 1GB file)?

No, it is not advisable. RSA is inefficient for large data because:

1. **Performance cost:** RSA modular exponentiation is much slower than symmetric ciphers like AES, especially for gigabyte-scale payloads.
2. **Size limits per RSA operation:** RSA can only encrypt small blocks (smaller than key size minus padding overhead), so large files must be split into many chunks, which is impractical and error-prone.
3. **Padding and security constraints:** Secure RSA requires padding schemes (like OAEP), which further reduce per-block payload size.
4. **Poor scalability:** Encrypting 1GB directly with RSA would be extremely slow and resource-heavy compared with modern symmetric encryption.

Best practice:

1. Generate a random symmetric key (for example AES key).
2. Encrypt the 1GB file with AES (fast).
3. Encrypt only the AES key with RSA.
4. Send RSA-encrypted key + AES-encrypted file.

This hybrid approach is what TLS and most secure systems use.

Part 5 – MITM Attack

```
PS C:\Users\andre\GitHub\IoT> cd "C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.3 RSA Protected Socket\Problem"; c:\Users\andre\GitHub\IoT\.venv\Scripts\python.exe .\Server.py -p 2345
Server is listening to port 2345
=====
waiting for connection
Got connection from ('127.0.0.1', 64788)
server chooses key: 4245336617104853559
Client msg: This is the client. Let's follow your selected key to communicate!
=====
waiting for connection
[]
```

```
(.venv) PS C:\Users\andre\GitHub\IoT> cd "C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.3 RSA Protected Socket\Problem"; python.exe .\Client.py -p 2345
Trying to connect to 127.0.0.1:2345
Server's key: 4245336617104853559
server msg: Hello, this is the server.
(.venv) PS C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.3 RSA Protected Socket\Problem> []
```

```
(.venv) PS C:\Users\andre\GitHub\IoT> cd "C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.3 RSA Protected Socket\Problem"; python.exe .\Client.py -p 12345
Trying to connect to 127.0.0.1:12345
Server's key: 1418391947889157059
server msg: Hello, this is the server.
(.venv) PS C:\Users\andre\GitHub\IoT\LABS\LAB6\Lab6.3 RSA Protected Socket\Problem> []
```

How can the MITM attack be defended against?

1. **Authenticate public keys:** Use digital certificates signed by a trusted CA (TLS/PKI), or pre-shared trusted public keys.
2. **Use TLS instead of raw sockets:** TLS provides server authentication, key exchange integrity, and encrypted channels that prevent silent key substitution.
3. **Add certificate/public-key pinning:** The client accepts only the expected server key fingerprint, so a fake key is rejected.
4. **Use mutual authentication (optional but stronger):** Require both client and server certificates so each side verifies the other.
5. **Add handshake integrity checks:** Use signed key exchange or authenticated key agreement (for example ECDHE + signatures) so tampering is detectable.

Code

6.1 Client

```
s = socket.socket()

# connect to server with input ip and port
s.connect((ip, port))

# message transmission
# send "Hello Server" to the server
tmsg = "Hello Server"
s.send(tmsg.encode())

# receive server's reply (a bytearray)
rmsg_byte = s.recv(1024) #< change to an appropriate function
rmsg = rmsg_byte.decode()
print("Server: ", rmsg)

# structured data transmission
# send sdata to the server (hint: use json to transfer back and forth between
structured data and bytes)
# public key of RSA with p:2127836173 q:2167131101 d:855599238897129473
tdata = {'N': 4611299948341116473, 'e': 65537}
tdata_byte = json.dumps(tdata).encode() #< python dictionary to byte
conversion.
#< you can use your way to do the conversion. For example,
json.dump, or the "to_bytes" function
s.send(tdata_byte) #< send the byte array to the server

# receive server's data (check Server.py for detail)
rdata_byte = s.recv(4096) #< receive byte array from the socket
c = json.loads(rdata_byte.decode())['c'] #< interpret the byte array
print("Server: ", c) #< print the cipher sent by the server
```

6.1 Server

```
s = socket.socket()

# bind to input port
s.bind("", port)
print("Server is listening to port %d" %(port))
#put at most 2 connections in backlog (or say 3 connections at most). Refuse more
connections
s.listen(2)

msg = b''
addr=''
while True:
    print("="*30)
    print("waiting for connection")
    # Establish connection with client, where c is the connection handler and addr
    is the client's ip
    c, addr = s.accept()
    print ('Got connection from', addr)

    ## message transmission
    # receive message from the client
    rmsg_bytes = c.recv(1024)          #< receive the welcome message but in byte
array
    rmsg = rmsg_bytes.decode()         #< convert byte array to char string
    print("client: ", rmsg)

    # reply the client with tmsg
    tmsg = 'Hello, this is the server'
    c.send(tmsg.encode())
    ## data transmission
    rdata = json.loads(c.recv(4096).decode()) #< necessary codes to process the
RSA parameter from the client
    e = rdata['e']                     #< interpret the e from the received data
    N = rdata['N']                     #< interpret N
    print("client e: ", e)
    print("client N: ", N)

    # We will focusing on generating the key in the next section.
    # Here, we temporarily use the cipher that is provided
    # reply
    tdata = {'c':2438923152802212086} # c is a RSA cipher
    c.send(json.dumps(tdata).encode())
    # Close the connection with the client
    c.close()
```

6.2 RSA_Offline

```
gap = random.randint(2**27, 2**29) #

p = npkg.find_prime_smaller_than_k(2**31 - gap) #< the first prime number by seeking
down from 2**31-gap
q = npkg.find_prime_greater_than_k(2**31 + gap) #< the first prime number by seeking
up from 2**31+gap
e = 65537 # 65537 is a typical selection of e

# follow the RSA algorithm
N = p * q
d = npkg.mult_inv_mod_N(e, (p - 1) * (q - 1))

print("Alice choose:\np:{p}\nq:{q}\nN:{N}\ne:{e}\nd:{d}".format(p=p, q=q, N=N, e=e,
d=d))
"""

Step 2:
Alice sends (N, e) to Bob

Notes:
For this lab, you DON'T need to write anything here. This will be integrated into the
socket example in the next section.
"""

# This task is done magically in this section.
"""

Step 3:
Bob uses the received (N, e)
"""

m = random.randint(0, N) # a randomly chosen message in GF(N). You can replace this
line with any desinated message in the future
c = npkg.exp_mod(m, e, N) # encrypt m
print("Bob chooses to encrypt plaintext:", m)
print("Bob sends ciphertext      :", c)
"""

Step 4:
Bob sends the encrypted message to Alice
For this lab, you DON'T need to write anything here. You can think about adding socket
communication codes here in the future.

Step 5:
Alice uses her private key d to decrypt Bob's ciphertext.

Ideally, m = m_decrypt
"""

m_decrypt = npkg.exp_mod(c, d, N) #< decrypt c with Alice's private keys
print("Alice decrypts Bob's msg and get:", m_decrypt)
```

6.3 RSA Protected Socket Client

```
#####Fill your RSA implementation here from Lab6.2#####
```

```
def RSA_keygen():
    gap = random.randint(2**27, 2**29)

    # your code to implement RSA_keygen
    p = npkg.find_prime_smaller_than_k(2**31 - gap)
    q = npkg.find_prime_greater_than_k(2**31 + gap)
    e = 65537
    N = p * q
    d = npkg.mult_inv_mod_N(e, (p - 1) * (q - 1))

    return e, d, N
```

```
def RSA_decrypt(c, d, N):

    # your code to implement RSA_decrypt
    m = npkg.exp_mod(c, d, N)

    return m
```

```
#####
```

```
s = socket.socket()

# connect to server with input ip and port
s.connect((ip, port))

# generate public and private keys
e, d, N = RSA_keygen()

# send public keys to the server
tdata = {'N': N, 'e': e}
# send tdata to the server.
s.send(json.dumps(tdata).encode())

# receive server's cipher (an integer)
rdata_bytes = s.recv(4096)
rdata = json.loads(rdata_bytes.decode())
c = rdata['c']

# decrypt cipher
key = RSA_decrypt(c, d, N)
print("Server's key:", key)
```



```

    # for real applications, "m" can be treated as an initial key to encrypt the
    following transmission by choosing DES, tri-DES, AES, etc.
    # encrypt following message using provided bytearray_xor()
    plaintext = "This is the client. Let's follow your selected key to communicate!"
    ciphertext_bytes = bytearray_xor(key, plaintext.encode()) #< encrypt the message
    based on the key

    # send ciphertext_bytes to the server
    s.send(ciphertext_bytes)

    # receive server's reply
    rciphertext_bytes = s.recv(1024)
    rplaintext_bytes = bytearray_xor(key, rciphertext_bytes)
    print("server msg:", rplaintext_bytes.decode())

```

6.3 RSA Protected Socket Server

```

    # put at most 2 connections in backlog (or say 3 connections at most). Refuse
    more connections
    s.listen(2)

    while True:
        print("="*30)
        print("waiting for connection")

        # Establish connection with client, where c is the connection handler and addr
        is the client's ip
        c, addr = s.accept()
        print('Got connection from', addr)

        # receive client's public keys 'e' and 'N'
        rdata_bytes = c.recv(4096)
        # use json to parse the receive data
        rdata = json.loads(rdata_bytes.decode())
        e, N = rdata['e'], rdata['N']

        # randomly choose a key
        key = random.randint(1, N - 1)
        print("server chooses key:", key)

        # encrypt m
        tdata = {'c':RSA_encrypt(key, e, N)}
        # send tdata to the client
        c.send(json.dumps(tdata).encode())

        # for real applications, "m" can be treated as an initial key to encrypt the
        following transmission by choosing DES, tri-DES, AES, etc.
        ciphertext_bytes = c.recv(1024)

```

```

plaintext_bytes = bytearray_xor(key, ciphertext_bytes)
print("Client msg:", plaintext_bytes.decode())

# server sends a reply message
plaintext = "Hello, this is the server."
ciphertext_bytes = bytearray_xor(key, plaintext.encode())
c.send(ciphertext_bytes)
# Close the connection with the client
c.close()

```

6.4 Mallory

```

import random

#####Fill your RSA implementation here from Lab5.2#####

def RSA_encrypt(m, e, N):
    c = npkg.exp_mod(m, e, N)
    return c

def RSA_keygen():
    gap = random.randint(2**27, 2**29)
    p = npkg.find_prime_smaller_than_k(2**31 - gap)
    q = npkg.find_prime_greater_than_k(2**31 + gap)
    e = 65537
    N = p * q
    d = npkg.mult_inv_mod_N(e, (p - 1) * (q - 1))
    return e, d, N

def RSA_decrypt(c, d, N):
    m = npkg.exp_mod(c, d, N)
    return m

#

def bytearray_xor(key, msg_bytes):
    key_bytearray = key.to_bytes(math.ceil(math.log2(key)), byteorder="big")

    key_len, msg_len = len(key_bytearray), len(msg_bytes)

    return bytearray(msg_bytes[i] ^ key_bytearray[i % key_len] for i in range(msg_len))

```

```

##### write your code after this line#####

# put no connections in backlog (or say 3 connections at most). Refuse more
connections
s_clt.listen(0)

while True:
    print("="*30)
    print("waiting for victim to connect")
    # Establish connection with client.

    """deal with the connected victim"""
    c, addr = s_clt.accept()
    print('Got connection from victim', addr)

    # receive client's public keys 'e' and 'N'
    rdata_bytes = c.recv(4096)
    rdata = json.loads(rdata_bytes.decode())

    e4vic, N4vic = rdata['e'], rdata['N']

    """build up the connection to the real server"""
    s_svr = socket.socket()
    # connect to the real server
    s_svr.connect((server_ip, server_port))

    # generate a new set of keys to communicate with the server
    e4svr, d4svr, N4svr = RSA_keygen() # generate the (e, d, N) for the server

    # send public keys to the server
    tdata = {'N': N4svr, 'e': e4svr}
    s_svr.send(json.dumps(tdata).encode()) # send the public keys to the server

    # receive server's cipher
    rdata_bytes = s_svr.recv(4096)
    rdata = json.loads(rdata_bytes.decode())
    c4svr = rdata['c']

    # decrypt out server's selected key
    key = RSA_decrypt(c4svr, d4svr, N4svr) # decrypt the key chosen by the server
    print("Server's key:", key)

    # encrypt key and send to the victim (client)
    tdata = {'c': RSA_encrypt(key, e4vic, N4vic)}
    # send tdata to the victim
    c.send(json.dumps(tdata).encode())

```

```
        """So far, Mallory has acquired the key and set up connections to both the real
server and the victim client."""
        # get the victim's message, decrypt and then encrypt using server's key and
send out
        ciphertext_from_victim_bytes = c.recv(1024)
        plaintext_from_victim_bytes = bytearray_xor(
            key, ciphertext_from_victim_bytes)
        print("Victim -> Server:", plaintext_from_victim_bytes.decode())
        # send to the server
        s_svr.send(ciphertext_from_victim_bytes)

        # receive server's reply, decrypt the ciphertext locally to get the message
        ciphertext_from_server_bytes = s_svr.recv(1024)
        plaintext_from_server_bytes = bytearray_xor(
            key, ciphertext_from_server_bytes)
        print("Server -> Client:", plaintext_from_server_bytes.decode())
        # send client the received ciphertext
        c.send(ciphertext_from_server_bytes)

        # Close the connection with the client and the server
        c.close()
        s_svr.close()
```